

Sports Results Prediction Platform

Technical Specification, Phased Delivery Plan & Claude Code Prompts

Document	Version 1.2 — September 2026 (EPL + La Liga, prompts for all phases)
Scope (v1)	Football (soccer): English Premier League and Spanish La Liga. 1X2 match-outcome probabilities and scoreline distributions. Architecture is multi-league and multi-sport ready.
Runtime	Docker Compose (Linux containers), Python 3.12
Build approach	Incremental phases, each with automated acceptance tests, implemented with Claude Code

Contents

- 1. Goals and non-goals
- 2. System architecture
- 3. Technology stack
- 4. Data sources
- 5. Data model
- 6. Data pipeline (fetch & preprocess)
- 7. Modeling specification
- 8. Evaluation & backtesting specification
- 9. API specification
- 10. Non-functional requirements
- 11. Phased delivery plan with tests
- 12. Claude Code: prompts for every phase

1. Goals and non-goals

Goals

- Automatically fetch, store and preprocess historical and upcoming match data for the **Premier League (EPL)** and **La Liga** inside Docker containers.
- Produce **calibrated probabilities** (home / draw / away) and full scoreline distributions for upcoming matches.
- Combine statistical models (ratings, Poisson/Dixon-Coles, Bayesian hierarchical) with machine learning (gradient boosting) and ensembling.
- Provide an honest, leakage-free walk-forward backtesting framework and compare every model against the betting-market baseline.
- Serve predictions through a REST API and a simple dashboard; track every experiment in MLflow.

Non-goals for v1

- Live in-play prediction, player-level props, other leagues or cups (Champions League, FA Cup, Copa del Rey), or other sports. The design must allow them later.
- Automated bet placement. The platform is analytical; any betting use is outside scope.
- Horizontal scaling / Kubernetes. A single host running Docker Compose is sufficient.

Primary success metric: Ranked Probability Score (RPS) and log loss on out-of-sample seasons, reported alongside the de-margined closing-odds baseline. Accuracy is a secondary, informational metric only.

2. System architecture

All services run under one `docker-compose.yml` on a shared internal network. Batch-style services (ingest, pipeline, train) share one Python image with different entrypoints.

Service	Responsibility	Image / Port
db	PostgreSQL 16: raw payloads, clean tables, features, predictions. Named volume for persistence.	postgres:16 / 5432 (internal)
ingest	Scheduled fetchers per source (APScheduler). Writes raw payloads, then triggers processing.	app image / —
pipeline	Cleaning, validation (pandera), entity resolution, point-in-time feature building.	app image / —
train	Model fitting, walk-forward backtests, calibration; logs to MLflow; registers models.	app image / —
mlflow	Experiment tracking and model registry (Postgres backend, local artifact volume).	mlflow image / 5000
api	FastAPI: predictions, fixtures, ratings, backtest reports, health.	app image / 8000
dashboard	Streamlit UI: upcoming predictions, calibration plots, backtest comparison.	app image / 8501

Data flow

```
sources --> [ingest] --> raw_payloads (immutable JSON/CSV blobs)
      |
      +--> [pipeline] --> teams / matches / odds (clean, validated)
      |
      +--> [pipeline] --> features (point-in-time, keyed by match_id + feature_version)
      |
      +--> [train] ---> MLflow runs + registered model
      |
      +--> [train] ---> predictions (match_id, model_version, probs, created_at)
      |
      +--> [api] + [dashboard] read predictions / reports
```

Repository layout

```
sports-predictor/
  CLAUDE.md                # rules for Claude Code (see section 12)
  docker-compose.yml        Dockerfile Makefile pyproject.toml uv.lock .env.example
  alembic/                  # DB migrations
  config/                   # settings.yaml (competitions: EPL + LaLiga, seasons, model params)
  src/predictor/
    config.py db.py logging.py
    ingestion/          # one module per source: base.py, football_data_uk.py, ...
    processing/          # clean.py, validate.py (pandera schemas), entities.py
    features/            # elo.py, form.py, schedule.py, market.py, builder.py
    models/              # base.py, baselines.py, elo_model.py, poisson.py,
                        # dixon-coles.py, bayesian.py, gbm.py, ensemble.py, calibration.py
    evaluation/          # metrics.py, backtest.py, reports.py
    simulation/          # monte_carlo.py (season tables)
    api/                 # main.py, routers/, schemas.py
    dashboard/           # app.py
    cli.py               # Typer CLI: ingest, process, features, train, backtest, predict
  tests/
    unit/ integration/ e2e/ fixtures/ # recorded source files; no network in tests
  data/raw/              # gitignored local cache of raw downloads
```

3. Technology stack

Area	Choice	Notes
Language / deps	Python 3.12, uv, pyproject.toml	Lockfile committed; pinned versions.
Data	pandas (or polars), SQLAlchemy 2.x, Alembic, psycopg	Migrations for every schema change.
Validation	pandera, pydantic v2	Schema checks on ingest and on feature tables.
Statistics	numpy, scipy, statsmodels	Poisson GLM, optimisation for Dixon-Coles.
Bayesian	PyMC (or CmdStanPy)	Hierarchical team-strength model.
ML	scikit-learn, LightGBM, Optuna	Tuning only inside walk-forward folds.
Tracking	MLflow	Params, metrics, artifacts, model registry.
Orchestration	APScheduler (upgrade path: Prefect)	Keep it simple in v1.
Serving / UI	FastAPI + Uvicorn, Streamlit	OpenAPI docs auto-generated.
Quality	pytest, hypothesis, ruff, mypy, pre-commit	GitHub Actions CI runs all checks.

4. Data sources

Verify current terms of use, licensing and rate limits before integrating any source. Every source is implemented behind a common Source interface so sources can be swapped.

Source	Use	Phase
football-data.co.uk (CSV)	Historical results, match stats and closing odds. Primary v1 source: files E0 (Premier League) and SP1 (La Liga), one CSV per season.	1
football-data.org or API-Football (REST, API key)	Upcoming fixtures and near-real-time results for both leagues (competition codes differ per provider; map them in settings.yaml).	7
StatsBomb open data (JSON)	Event data / xG for research features (limited coverage).	8 (optional)
Understat / FBref-style xG	Team xG features. Only if terms permit; throttle and cache.	8 (optional)

5. Data model (PostgreSQL)

Table	Key columns	Notes
raw_payloads	id, source, resource, fetched_at (UTC), checksum, payload (bytea/jsonb)	Immutable. Unique on (source, resource, checksum).
competitions / seasons	competition_id, sport, country, name / season_id, label, start, end	Sport column keeps it multi-sport.
teams, team_aliases	team_id, canonical_name / alias, source, team_id	Entity resolution: every source name maps to one team_id.
matches	match_id, season_id, kickoff_utc, home_id, away_id, home_goals, away_goals, status	Unique on (season_id, home_id, away_id, kickoff date). season_id belongs to one competition.
match_stats	match_id, team_id, shots, shots_on_target, corners, cards, xg (nullable)	Only used as lagged inputs.
odds	match_id, bookmaker, market, captured_at, price_home/draw/away, is_closing	Closing odds used for baseline only.
features	match_id, feature_version, computed_at, as_of_utc, values (jsonb or wide)	as_of_utc must be < kickoff_utc.
predictions	match_id, model_name, model_version, p_home, p_draw, p_away, score_matrix, created_at	Append-only history.
backtest_results	run_id, model_name, season_id, n, rps, log_loss, brier, ece	Mirrors MLflow for dashboard queries.

6. Data pipeline (fetch & preprocess)

6.1 Ingestion requirements

- **Idempotent:** re-running any fetch produces no duplicate rows (checksum dedupe on raw, upserts on clean tables).
- **Resilient:** retries with exponential backoff and jitter; configurable per-source rate limit; timeouts.
- **Raw first:** store the exact payload before parsing, so processing can be re-run without refetching.
- **Secrets** (API keys) only via environment variables from .env; never committed.
- **Tests never hit the network:** use recorded fixture files in tests/fixtures/.

6.2 Preprocessing steps

- Parse dates to UTC using each competition's local timezone (Europe/London for EPL, Europe/Madrid for La Liga) before converting; handle daylight-saving changes.
- Normalise team names via team_aliases, keyed by competition (e.g. 'Ath Madrid' / 'Atletico Madrid' / 'Atlético de Madrid'; 'Betis' / 'Real Betis'; 'Sociedad' / 'Real Sociedad'). Unknown aliases fail loudly and are logged for review. Handle accents consistently (store canonical names in UTF-8, match on a normalised key).
- Validate with pandera: non-negative goals, home != away, one row per fixture, odds > 1.0, plausible kickoff times.
- Handle postponed / abandoned / awarded matches with an explicit status; exclude from training where appropriate.
- Compute de-margined market probabilities (proportional normalisation and Shin's method) for the baseline.

6.3 Point-in-time feature rules (leakage prevention)

- Each feature row stores as_of_utc; the builder may only read data with timestamps strictly before kickoff.

- Features are computed by replaying history in chronological order (ratings updated only after a match is played).
- Initial feature set: Elo / pi-ratings (pre-match), rolling goals for/against and shots over last N matches (exponentially weighted), rest days, matches in last 14 days, home/away splits, season stage, promoted-team flag, pre-match market probabilities (optional, flagged).
- Feature sets are versioned (feature_version) so models are reproducible.

7. Modeling specification

All models implement a common interface so the backtester treats them identically:

```
class MatchModel(Protocol):
    name: str
    def fit(self, matches: pd.DataFrame, features: pd.DataFrame | None) -> "MatchModel": ...
    def predict_proba(self, fixtures: pd.DataFrame, features: pd.DataFrame | None) -> pd.DataFrame:
        # returns columns: match_id, p_home, p_draw, p_away (rows sum to 1)
    def predict_scores(self, fixtures, features=None, max_goals: int = 10) -> dict[int, np.ndarray]:
        # optional: scoreline probability matrix per match
```

#	Model	Specification
M0	Baselines	(a) historical home/draw/away frequencies; (b) de-margined closing odds (benchmark to compare against, not a model to deploy).
M1	Elo	$E_{\text{home}} = 1 / (1 + 10^{-(R_h + \text{HFA} - R_a)/400})$; update $R' = R + K \cdot G \cdot (S - E)$ with goal-difference multiplier G . Map rating difference to 1X2 via ordered logistic regression. Tune K , HFA , season regression-to-mean.
M2	Poisson	Goals $\sim \text{Poisson}(\lambda)$; $\log \lambda_{\text{home}} = \mu + h + \text{att}_{\text{home}} + \text{def}_{\text{away}}$, $\log \lambda_{\text{away}} = \mu + \text{att}_{\text{away}} + \text{def}_{\text{home}}$. Sum-to-zero constraints on att/def. Outcome probabilities from the scoreline matrix.
M3	Dixon-Coles	M2 plus the low-score dependence correction $\tau(x, y; \rho)$ for scores 0-0, 1-0, 0-1, 1-1, and time-decay weights $w = \exp(-\xi \cdot t)$. Fit by weighted maximum likelihood (scipy.optimize); tune ξ by walk-forward RPS.
M4	Bayesian hierarchical	PyMC model of M2/M3 with $\text{att}, \text{def} \sim \text{Normal}(0, \sigma_{\text{att}})$, $\sigma \sim \text{HalfNormal}$: partial pooling helps promoted teams. Posterior predictive gives uncertainty. Optional dynamic (random-walk) team strengths.
M5	Gradient boosting	LightGBM multiclass on the feature store, with M1-M3 outputs as features. Early stopping and Optuna tuning strictly within training folds.
M6	Ensemble + calibration	Stacking (multinomial logistic regression on out-of-fold probabilities) or weighted average optimised on RPS; then isotonic or temperature calibration fit on a held-out, later-in-time slice.
S1	Season simulation	Monte Carlo (e.g. 10,000 runs) of remaining fixtures using the scoreline matrices: title, top-4 and relegation probabilities.

7.1 Handling two leagues

- **Separate team-strength parameters per league** for M1-M4. Teams from the EPL and La Liga never play each other in league matches, so a joint model cannot identify relative strength between leagues. Fit one model per competition behind the same interface (a per_competition wrapper).
- **League-specific parameters:** home advantage, average goal rate μ , Dixon-Coles ρ and time-decay ξ are fitted per league (La Liga has historically had fewer goals and more draws than the EPL; let the data decide).
- **Promoted teams:** initialise their rating from the relegated teams' average in that league (Elo) or rely on hierarchical pooling (M4).
- **M5/M6 (LightGBM, ensemble)** may train on both leagues together with a competition categorical feature; this gives more data. Compare against per-league training in the backtest and keep whichever scores better.
- Season simulation (S1) runs per league with that league's rules: top 4 to Champions League and bottom 3 relegated in both; tie-breakers differ (EPL: goal difference; La Liga: head-to-head first). Implement tie-breakers per competition.

8. Evaluation & backtesting specification

Walk-forward protocol. For each test season s : train on all matches before the season starts, then predict the season in chronological order. Optionally refit every matchweek (models that update online, like Elo, update after every match). Random K-fold cross-validation is forbidden.

Metrics (per season and pooled):

- **RPS** (primary; respects outcome order $H > D > A$):
$$RPS = 1/(r-1) \cdot \sum_{i=1..r-1} \left(\sum_{j \leq i} (p_j - o_j) \right)^2$$
, with $r = 3$ outcomes; lower is better.
- **Log loss** and **Brier score** (multiclass).
- **Calibration**: expected calibration error (ECE, 10 bins) plus a reliability diagram per outcome.
- **Per league**: every metric is reported separately for EPL and La Liga, plus pooled.
- **Market gap**: model RPS minus de-margined closing-odds RPS on the same matches (negative would mean beating the market; expect it to be positive).
- **Uncertainty**: bootstrap 95% confidence intervals on metric differences between models; a model is only 'better' if the interval excludes zero.

Promotion rule. A new model version is registered as *Production* only if its pooled walk-forward RPS beats the current Production model with a bootstrap CI excluding zero, it is not significantly worse in either league, and its ECE does not get worse by more than 0.01.

9. API specification (FastAPI)

Method & path	Description
GET /health	Liveness + DB connectivity + model registry reachable.
GET /competitions	Lists supported competitions: EPL, LALIGA.
GET /fixtures?competition=&from=&to=	Upcoming fixtures with kickoff times (UTC). competition = EPL LALIGA.
GET /predictions/{match_id}	Latest Production-model probabilities, most-likely scores, model version, created_at.
GET /predictions?competition=&matchweek=	Batch predictions.
GET /ratings?competition=&as_of=	Team ratings (Elo, attack/defence) as of a date.
GET /simulations/{competition}/{season}	Monte Carlo table probabilities.
GET /backtests?model=	Stored walk-forward metrics per season.
POST /admin/refresh (token)	Trigger ingest + process + predict for upcoming fixtures.

All responses are pydantic models; the API is read-only except `/admin/*`, which requires a bearer token from the environment.

10. Non-functional requirements

Area	Requirement
Reproducibility	Fixed random seeds; feature_version + model params + git SHA logged to MLflow; locked dependencies; the same command reproduces a backtest.
Configuration	All tunables in config/settings.yaml, overridable by env vars (pydantic-settings). No magic numbers in code.
Logging	Structured JSON logs (structlog), one run_id per pipeline execution.
Testing	Unit + integration (Postgres via testcontainers or compose) + e2e smoke. Target $\geq 80\%$ coverage on src/predictor (excluding dashboard).
Code quality	ruff (lint + format), mypy (strict on core modules), pre-commit, CI on every push.
Security	Secrets only in env; non-root containers; DB not exposed outside the compose network except in dev.
Performance	Full history for both leagues (about 30 seasons each) re-processes and rebuilds features in under 5 minutes on a laptop.
Data retention	Raw payloads kept forever (small); nightly pg_dump to a mounted backup volume.

11. Phased delivery plan with tests

Each phase ends with a demo command and automated acceptance tests. Do not start the next phase until every acceptance test passes and `make check` (lint + types + tests) is green.

Phase 0 — Foundations	<i>Skeleton that builds and runs.</i>
Deliverables	• Repo layout, pyproject, uv lock, Dockerfile (multi-stage, non-root), docker-compose with db + api + mlflow. • Settings module, structured logging, Alembic initialised, Typer CLI stub, Makefile, pre-commit, CI workflow, CLAUDE.md.
Acceptance tests	• <code>docker compose up -d</code> brings all services to healthy within 60 s. • <code>GET /health</code> returns 200 with db=ok (integration test). • <code>make check</code> passes: ruff, mypy, pytest (with at least a smoke test). • <code>alembic upgrade head</code> runs cleanly on an empty database.
Demo command	<code>make up && make check</code>
Phase 1 — Ingestion	<i>Historical EPL and La Liga data in the database, reproducibly.</i>
Deliverables	• Source interface + football-data.co.uk ingester for 10+ seasons of both leagues (E0, SP1). • <code>raw_payloads</code> table with checksum dedupe; retry/backoff/rate limit; CLI <code>ingest --source --competition --seasons</code>.
Acceptance tests	• Unit tests parse recorded CSV fixtures for both leagues (no network). • Idempotency test: running ingest twice yields identical row counts. • Retry test: mocked HTTP 500s then 200 succeeds; permanent 404 fails with a clear error. • Integration: every complete season of each league has 380 matches.
Demo command	<code>make ingest COMPETITIONS=EPL,LALIGA SEASONS=2014-2025</code>
Phase 2 — Processing & validation	<i>Clean, trustworthy tables.</i>
Deliverables	• Entity resolution (teams, team_aliases), cleaning to matches / match_stats / odds. • pandera schemas; de-margined market probabilities (proportional + Shin).
Acceptance tests	• Every alias resolves to exactly one team per competition; unknown alias raises and is logged; accented and unaccented spellings map to the same team. • Invariants in both leagues: home != away; goals >= 0; each team plays 38 matches per complete season; 20 teams per season; no match links teams from different competitions. • Timezone test: a La Liga 21:00 Madrid kickoff in winter and in summer converts to the correct UTC time. • De-margined probabilities sum to 1 (+/- 1e-9) and lie in (0, 1); hypothesis property tests. • Deliberately corrupted fixture file is rejected by validation.
Demo command	<code>make process</code>
Phase 3 — Point-in-time features	<i>Leakage-free, versioned feature store.</i>
Deliverables	• Chronological replay engine; Elo pre-match ratings; rolling / EWMA form; rest days; schedule congestion. • features table with feature_version and as_of_utc.
Acceptance tests	• Truncation leakage test: features for match m computed on the full history equal features computed on history cut at m's kickoff (for a random sample of 200 matches). • Every row has <code>as_of_utc < kickoff_utc</code>. • Hand-calculated golden test for Elo on a 5-match toy dataset. • League isolation: ratings of one league are unchanged when the other league's data is removed. • Rebuilding features twice gives byte-identical output (determinism).
Demo command	<code>make features</code>

Phase 4 — Backtesting & baselines	<i>Honest measuring stick before any real model.</i>
Deliverables	• Metrics module (RPS, log loss, Brier, ECE), walk-forward engine, bootstrap CIs, MLflow logging, report generation. • Baselines M0a (frequencies) and M0b (market).
Acceptance tests	• Metrics match hand-computed values on known examples (e.g. perfect forecast gives RPS 0). • Engine test: a model that tries to read future data raises an error (guarded data access). • Baseline backtest runs for 5+ test seasons per league and logs to MLflow. • Market baseline beats frequency baseline (sanity check that the plumbing is right).
Demo command	<code>make backtest MODEL=baseline_market</code>
Phase 5 — Statistical models	<i>Elo, Poisson, Dixon-Coles, Bayesian hierarchical.</i>
Deliverables	• M1-M4 behind the MatchModel interface, fitted per competition; scoreline matrices; per-league parameter tuning via walk-forward.
Acceptance tests	• Parameter recovery: simulate data from known attack/defence/p values; the fit recovers them within tolerance. • Probabilities sum to 1; score matrices sum to ~1 (truncation error < 1e-4). • Each model beats the frequency baseline on RPS in each league (bootstrap CI excludes 0). • Dixon-Coles is no worse than plain Poisson; results table logged to MLflow.
Demo command	<code>make backtest MODEL=dixon_coles</code>
Phase 6 — ML, ensemble & calibration	<i>Best achievable probabilities.</i>
Deliverables	• LightGBM on features + statistical model outputs (joint vs per-league training compared); stacking ensemble; calibration; model registry + promotion rule.
Acceptance tests	• Hyper-parameter search touches only training folds (test asserts fold boundaries). • Calibrated ensemble ECE <= uncalibrated ECE on the held-out slice. • Promotion rule implemented and unit-tested (promotes, rejects, ties). • Report: every model's RPS vs market gap per season and per league.
Demo command	<code>make train && make backtest MODEL=ensemble</code>
Phase 7 — Serving, automation & simulation	<i>Daily predictions available to users.</i>
Deliverables	• Upcoming-fixtures source, APScheduler jobs, prediction writer, full API, Streamlit dashboard, Monte Carlo season simulation.
Acceptance tests	• API contract tests for every endpoint (schemas, 404s, auth on /admin). • E2E smoke: fresh docker compose up -> ingest fixtures -> process -> features -> train -> predict -> API returns probabilities for the next matchweek of both leagues. • Simulation (both leagues): each team's finishing-position probabilities sum to 1; each position sums to 1 across teams; fixed seed reproduces results; tie-breaker unit tests (goal difference for EPL, head-to-head for La Liga). • Scheduler job test with frozen time (freezegun).
Demo command	<code>make e2e</code>

Phase 8 — Hardening & expansion	<i>Operate and grow.</i>
Deliverables	• Monitoring of live performance (rolling RPS on finished matches), data-drift checks, backups, alerts. • Add more leagues (e.g. Serie A, Bundesliga), xG features, and later other sports (new Source + models per sport).
Acceptance tests	• Live-vs-backtest RPS dashboard; alert when rolling RPS exceeds backtest CI. • Adding a third league requires only config + aliases + tie-breaker rule (test with a fixture league). • Backup restore drill documented and tested.
Demo command	<code>make monitor</code>

12. Claude Code: prompts for every phase

Before you start: create an empty git repo, copy this PDF into it as docs/SPEC.pdf (Claude Code can read PDFs), open the folder in Claude Code, switch to **plan mode**, and paste the Phase 0 prompt. Start a fresh session (or run /clear) for each following phase and paste its prompt. The same prompts are provided as a separate text file for easy copying.

12.1 Prompt for Phase 0 — Foundations

PHASE 0 - FOUNDATIONS

You are helping me build a sports results prediction platform. The full technical specification is in docs/SPEC.pdf. Read it completely before doing anything else.

Context

- v1 scope: English Premier League (EPL) and Spanish La Liga (LALIGA), 1X2 probabilities + scoreline distributions. Team-strength models are fitted per league (teams never meet across leagues in league play).
- Everything runs with Docker Compose. Python 3.12, uv, PostgreSQL 16, FastAPI, MLflow. The architecture must stay multi-league and multi-sport ready (sport column, Source and MatchModel interfaces) but build nothing else yet.
- We deliver in phases (spec section 11). Today we do PHASE 0 ONLY.

Step 1 - Create CLAUDE.md at the repo root with these non-negotiable rules:

- Work one phase at a time; never implement features from a later phase.
- No data leakage: features use only data with timestamps strictly before kickoff; never use random train/test splits; walk-forward validation only.
- Tests never touch the network; use recorded files in tests/fixtures/.
- Secrets only from environment variables (.env, with .env.example committed).
- Every tunable lives in config/settings.yaml, not in code. Competitions (EPL, LALIGA), their source codes, timezones and rules are config too; never hard-code a league name in logic.
- Store all times in UTC; convert from each league's local timezone.
- Type hints everywhere; ruff + mypy clean; every new module has tests.
- Schema changes only through Alembic migrations.
- Run `make check` before saying a task is done, and show me the output.
- If the spec is ambiguous, ask me instead of guessing.

Also summarise the architecture, repo layout and Makefile targets in CLAUDE.md.

Step 2 - Propose a plan for Phase 0 and wait for my approval. Phase 0 delivers:

- Repo layout from spec section 2, pyproject.toml managed by uv, lockfile.
- Multi-stage Dockerfile (non-root user) shared by api/ingest/pipeline/train.
- docker-compose.yml with db, api and mlflow (healthchecks, named volumes, internal network); ingest/pipeline/train/dashboard as stubs for later.
- config/settings.yaml with a competitions list (EPL: football-data.co.uk E0, Europe/London; LALIGA: SP1, Europe/Madrid), validated at startup.
- src/predictor: config.py (pydantic-settings reading settings.yaml + env), logging.py (structlog JSON), db.py (SQLAlchemy 2 engine/session), cli.py (Typer with stub commands), api/main.py with GET /health.
- Alembic initialised with an empty baseline migration.
- Makefile targets: up, down, logs, check (ruff + mypy + pytest), test, migrate.
- pre-commit config and a GitHub Actions CI workflow running `make check`.

Acceptance tests (all must pass):

1. `docker compose up -d` brings db, api and mlflow to healthy within 60 s.
2. GET /health returns 200 with {"status": "ok", "db": "ok"} (integration test).
3. `make check` passes: ruff, mypy, pytest (including a test that settings load both competitions and reject an invalid one).
4. `alembic upgrade head` runs cleanly on an empty database.

When done, show me: the file tree, the `make check` output, the curl output of /health, and a short list of anything you were unsure about.

12.2 Prompt for Phase 1 — Ingestion

PHASE 1 - INGESTION

Read CLAUDE.md and docs/SPEC.pdf sections 4, 5, 6.1 and 11 (Phase 1). Summarise what already exists that this phase builds on, then propose a plan and wait for my approval. Do not implement anything from Phase 2 or later.

Goal: historical EPL and La Liga data stored in the database, reproducibly.

Deliver:

- src/predictor/ingestion/base.py: an abstract Source interface (list_resources, fetch, returning raw bytes + metadata).
- ingestion/football_data_uk.py: fetches season CSVs for the codes configured per competition (E0, SP1). Season list, base URL, timeout, retry count and rate limit all come from settings.yaml.
- HTTP client with timeouts, exponential backoff with jitter on 5xx and timeouts, no retry on 4xx, and a per-source rate limiter.
- Alembic migration for raw_payloads (id, source, competition, resource, fetched_at UTC, checksum sha256, payload bytea), unique on (source, resource, checksum).
- Repository function that inserts raw payloads idempotently.
- Minimal parse step that loads the CSV rows into a staging table (stg_matches) exactly as they are (all columns as text). Real cleaning is Phase 2.
- CLI: `predictor ingest --source football\_data\_uk --competition EPL,LALIGA --seasons 2014-2025`, and a Makefile target `make ingest COMPETITIONS=EPL,LALIGA SEASONS=2014-2025`.
- Record 2 real season CSVs per league into tests/fixtures/football_data_uk/.

Acceptance tests (write them first and show them failing):

1. Unit: parsing the recorded fixtures for both leagues gives the expected row counts and column names (no network; the HTTP layer is mocked).
2. Idempotency: running ingest twice produces identical row counts in raw_payloads and stg_matches.
3. Retry: mocked 500, 500, 200 succeeds; a 404 fails immediately with a clear error naming the resource; timeouts are retried.
4. Integration (against the compose database): every complete season of each league has exactly 380 staging rows.
5. Checksum change: a modified payload for the same resource is stored as a new raw row, and the old one is kept (raw data is immutable).

Finish with: `make check` output, the demo command output (row counts per league and season), and a self-review: anything hard-coded that belongs in settings.yaml, and tests that could pass trivially.

12.3 Prompt for Phase 2 — Processing & validation

PHASE 2 - PROCESSING & VALIDATION

Read CLAUDE.md and docs/SPEC.pdf sections 5, 6.2 and 11 (Phase 2).
Summarise what exists, propose a plan, wait for approval. Nothing from Phase 3+.

Goal: clean, validated tables for both leagues.

Deliver:

- Migrations for competitions, seasons, teams, team_aliases, matches, match_stats and odds as in spec section 5 (with a sport column on competitions, and each season belonging to one competition).
- processing/entities.py: resolve team names through team_aliases, keyed by competition. Match on a normalised key (lowercase, accents stripped, extra whitespace removed). Seed aliases from config/team_aliases.yaml covering every team in the ingested seasons (e.g. "Ath Madrid" / "Atletico Madrid" / "Atlético de Madrid", "Betis" / "Real Betis", "Sociedad", "Man United", "Nott'm Forest"). Unknown names raise UnknownTeamError and are logged.
- processing/clean.py: staging -> clean tables. Build kickoff_utc from date + time in the competition's timezone (Europe/London, Europe/Madrid), handling daylight-saving. Where the time is missing (older seasons), use a configurable default local time and set a kickoff_time_estimated flag. Set a match status (played / postponed / abandoned / awarded).
- Odds: store per bookmaker; mark closing odds where the source provides them.
- processing/market.py: de-margined probabilities using proportional normalisation and Shin's method.
- processing/validate.py: pandera schemas for matches, match_stats and odds.
- Upserts, so re-processing is idempotent. CLI `predictor process` and `make process`.

Acceptance tests (write first, show failing):

1. Every alias resolves to exactly one team within its competition; accented and unaccented spellings map to the same team; an unknown alias raises.
2. Invariants, per league and complete season: 20 teams; 38 matches per team; 380 matches; home != away; goals >= 0; no match links teams from different competitions.
3. Timezone: a 21:00 Madrid kickoff in January is 20:00 UTC and in September is 19:00 UTC; the same check for 15:00 London.
4. Market probabilities: sum to 1 (tolerance 1e-9) and lie in (0, 1) (hypothesis property tests); Shin gives a less extreme favourite probability than proportional on a known example.
5. A deliberately corrupted fixture (negative goals, duplicate match, odds <= 1.0) is rejected by validation with a readable error.
6. Running process twice leaves every table unchanged.

Finish with: `make check` output, the demo output (table row counts per league and season, plus the list of canonical teams), and a self-review.

12.4 Prompt for Phase 3 — Point-in-time features

PHASE 3 - POINT-IN-TIME FEATURES

Read CLAUDE.md and docs/SPEC.pdf sections 6.3, 7.1 and 11 (Phase 3).
Summarise what exists, propose a plan, wait for approval. Nothing from Phase 4+.

Goal: a leakage-free, versioned feature store. This is the most important phase for correctness, so be conservative and test heavily.

Deliver:

- features/replay.py: a chronological replay engine. It walks matches in kickoff order and, for each match, exposes ONLY data with timestamp < kickoff (a guarded "history view" object that raises if asked for anything later). Matches with the same kickoff time must not see each other's results.
- features/elo.py: pre-match Elo per competition (K, home advantage, goal-difference multiplier, season-start regression to the mean, and the promoted-team initial rating come from settings.yaml, per competition). Ratings update only after a match is played.
- features/form.py: exponentially weighted and last-N rolling goals for and against, shots and shots on target (overall and home/away split).
- features/schedule.py: rest days, matches in the last 14 days, season stage (matchweek / 38), promoted-team flag.
- features/builder.py: assembles a wide feature row per match with match_id, feature_version, as_of_utc and computed_at. Writes to a features table (migration). Pre-match market probabilities are an optional, clearly flagged feature group, off by default.
- CLI `predictor features --version v1` and `make features`.

Acceptance tests (write first, show failing):

1. Truncation leakage test: for 200 random matches across both leagues, features computed from the full history equal features computed from a database truncated at that match's kickoff (exact equality).
2. Every feature row has as_of_utc < kickoff_utc.
3. Guarded history: accessing a result at or after kickoff raises LeakageError.
4. Elo golden test: a hand-calculated 5-match toy dataset gives exactly the expected ratings.
5. League isolation: EPL features are identical whether or not La Liga data is present, and vice versa.
6. Determinism: building features twice gives identical output (same hash).
7. Performance: full rebuild for both leagues finishes in under 5 minutes (mark as a slow test).

Finish with: `make check` output, demo output (feature row counts, a sample row for one EPL and one La Liga match, and the top/bottom 5 Elo ratings per league today), and a self-review focused specifically on leakage paths.

12.5 Prompt for Phase 4 — Backtesting & baselines

PHASE 4 - BACKTESTING & BASELINES

Read CLAUDE.md and docs/SPEC.pdf sections 7 (interface), 8 and 11 (Phase 4). Summarise what exists, propose a plan, wait for approval. Nothing from Phase 5+.

Goal: an honest measuring stick before any real model exists.

Deliver:

- models/base.py: the MatchModel protocol from spec section 7, plus a PerCompetition wrapper that fits and predicts one model per competition.
- evaluation/metrics.py: RPS, multiclass log loss, multiclass Brier score, ECE (10 bins) and reliability-diagram data. Vectorised with numpy.
- evaluation/backtest.py: a walk-forward engine. For each test season: fit on everything before the season starts, then predict in chronological order. Support optional refitting every N matchweeks (settings.yaml). The model only receives data through the guarded history view from Phase 3.
- evaluation/bootstrap.py: paired bootstrap 95% CI for the difference in a metric between two models on the same matches.
- models/baselines.py: M0a (historical home/draw/away frequencies, per league, from training data only) and M0b (de-margined closing odds; a benchmark, marked as not deployable).
- MLflow logging: params, per-league and pooled metrics per season, reliability plots as artifacts, git SHA and feature_version.
- backtest_results table (migration) mirroring the MLflow metrics.
- CLI `predictor backtest --model <name> --seasons 2019-2025` and `make backtest MODEL=<name>`. Also `make report`: a markdown/HTML comparison table of all models.

Acceptance tests (write first, show failing):

1. Metrics match hand-computed values: perfect forecast gives RPS 0; uniform forecast gives a known RPS; log loss and Brier checked on a 3-row example.
2. Leakage guard: a deliberately "cheating" model that reads the result of the match it predicts makes the engine raise LeakageError.
3. Fold boundaries: the latest training kickoff is always before the earliest test kickoff in each fold.
4. Both baselines run for 5+ test seasons per league and log to MLflow.
5. Sanity: the market baseline beats the frequency baseline on RPS in each league (bootstrap CI excludes 0). If not, the plumbing is wrong.

Finish with: `make check`, the report table (RPS, log loss, Brier and ECE per league and season for both baselines), and a self-review.

12.6 Prompt for Phase 5 — Statistical models

PHASE 5 - STATISTICAL MODELS

Read CLAUDE.md and docs/SPEC.pdf sections 7, 7.1, 8 and 11 (Phase 5).
Summarise what exists, propose a plan, wait for approval. Nothing from Phase 6+.

Goal: Elo, Poisson, Dixon-Coles and Bayesian hierarchical models, each fitted per competition, behind the MatchModel interface.

Deliver:

- models/elo_model.py (M1): uses Phase 3 Elo ratings; maps the rating difference to H/D/A with an ordered logistic regression fitted on training data.
- models/poisson.py (M2): independent Poisson with attack, defence, home advantage and intercept; sum-to-zero constraints. Returns the scoreline matrix (max goals from settings, default 10) and derived 1X2.
- models/dixon-coles.py (M3): M2 + the tau low-score correction (ρ) + time-decay weights $\exp(-\xi * \text{days}/365)$. Weighted MLE with `scipy.optimize` (analytic gradients if practical). Tune ξ per league by walk-forward RPS on training seasons only.
- models/bayesian.py (M4): PyMC hierarchical Poisson with partial pooling of attack and defence; posterior predictive scoreline matrix. Use a fixed seed and settings-driven sampler settings. Keep runtime reasonable (document it); mark its backtest as a slow job.
- Promoted teams: sensible priors or initial values (spec 7.1).
- Register all models so ``make backtest MODEL=<name>`` works, and extend the report.

Acceptance tests (write first, show failing):

1. Parameter recovery: simulate 3 seasons from known attack, defence, home advantage and ρ ; M2/M3 recover them within stated tolerances; M4 posterior means are within tolerance.
2. Every probability row sums to 1 ($1e-9$); score matrices sum to 1 minus a truncation error $< 1e-4$.
3. Symmetry: two identical teams with home advantage 0 give $p_{\text{home}} == p_{\text{away}}$.
4. Each model beats the frequency baseline on RPS in each league (bootstrap CI excludes 0).
5. Dixon-Coles is not worse than Poisson on pooled RPS.
6. Refitting with the same seed and data gives identical parameters.

Finish with: ``make check``, the updated report (all models vs market per league and season), fitted home advantage / ρ / ξ per league, and a self-review.

12.7 Prompt for Phase 6 — ML, ensemble & calibration

PHASE 6 - MACHINE LEARNING, ENSEMBLE & CALIBRATION

Read CLAUDE.md and docs/SPEC.pdf sections 7, 8 and 11 (Phase 6).
Summarise what exists, propose a plan, wait for approval. Nothing from Phase 7+.

Goal: the best achievable calibrated probabilities, and a model registry.

Deliver:

- models/gbm.py (M5): LightGBM multiclass on the Phase 3 features plus out-of-fold outputs of M1-M3 (generated inside the walk-forward, never in-sample). Two variants: per-league, and joint with a competition categorical feature. Early stopping on a time-ordered validation slice.
- Hyperparameter tuning with Optuna, strictly inside training folds (nested time-based validation). Trial count from settings.
- models/ensemble.py (M6): stacking with multinomial logistic regression on out-of-fold probabilities, and an RPS-optimised weighted average; keep whichever backtests better.
- models/calibration.py: isotonic and temperature scaling, fitted on the latest part of the training window only.
- MLflow model registry and a promotion rule (spec section 8): promote to Production only if pooled RPS beats the current Production model with a bootstrap CI excluding 0, it is not significantly worse in either league, and ECE does not get worse by more than 0.01.
- `make train` trains and registers candidates; `make promote` applies the rule.

Acceptance tests (write first, show failing):

1. Tuning and calibration never see test-fold rows (assert on the timestamps of every fitted object).
2. Stacking inputs are out-of-fold (a test that injects in-fold predictions fails).
3. Calibrated ECE $\leq$ uncalibrated ECE on the held-out slice.
4. Promotion rule unit tests: promotes a clearly better model; rejects a worse model; rejects when one league is significantly worse; handles ties.
5. Report shows every model's RPS vs market gap per league and season, plus joint vs per-league GBM.

Finish with: `make check`, the report, which model is now Production and why, feature importances, and a self-review focused on leakage via stacking and tuning.

12.8 Prompt for Phase 7 — Serving, automation & simulation

PHASE 7 - SERVING, AUTOMATION & SIMULATION

Read CLAUDE.md and docs/SPEC.pdf sections 4, 7 (S1), 7.1, 9 and 11 (Phase 7). Summarise what exists, propose a plan, wait for approval. Nothing from Phase 8.

Goal: daily predictions for both leagues, available through the API and the dashboard.

Deliver:

- ingestion/fixtures_api.py: upcoming fixtures and recent results from football-data.org or API-Football (choose one and explain; API key from env; competition codes mapped in settings.yaml). Recorded fixtures for tests.
- A prediction writer: loads the Production model, builds features for upcoming fixtures, and writes to the predictions table (append-only, with model_version and created_at).
- simulation/monte_carlo.py: simulates the remaining season N times (settings, default 10,000) from scoreline matrices; outputs title, top-4, European places and relegation probabilities, plus the finishing-position distribution. Tie-breakers per competition: EPL points > goal difference > goals scored > head-to-head; La Liga points > head-to-head points > head-to-head goal difference > goal difference > goals scored. Keep the rules in config and verify them against the current official regulations.
- Scheduler service (APScheduler): daily ingest -> process -> features -> predict -> simulate, at times set in settings.yaml; the job is idempotent.
- The full API from spec section 9 (pydantic schemas, a competition parameter EPL | LALIGA, 404s, bearer token on /admin/*).
- Streamlit dashboard: league selector; upcoming matches with probabilities and most likely scores; simulated table; model performance (reliability plot, rolling RPS vs market).
- `make e2e` runs the whole chain on recorded fixtures.

Acceptance tests (write first, show failing):

1. API contract tests for every endpoint: schema, filtering by competition, 404 for an unknown match, 401 without a token on /admin/refresh.
2. E2E smoke: from a fresh `docker compose up` with recorded data, running the chain produces probabilities for the next matchweek of both leagues via the API.
3. Simulation: each team's position probabilities sum to 1; each position sums to 1 across teams; a fixed seed reproduces the output; tie-breaker unit tests for both leagues using hand-made tables.
4. Scheduler test with frozen time (freezegun): the job fires at the configured time and running it twice creates no duplicates.
5. Predictions are never made for matches that have already kicked off.

Finish with: `make check`, `make e2e` output, screenshots or a description of the dashboard, sample API responses for one EPL and one La Liga match, and a self-review.

12.9 Prompt for Phase 8 — Hardening & expansion

PHASE 8 - HARDENING & EXPANSION

Read CLAUDE.md and docs/SPEC.pdf sections 10 and 11 (Phase 8).
Summarise what exists, propose a plan, wait for approval.

Goal: operate the system reliably, and make it easy to grow.

Deliver:

- Live monitoring: after each matchweek, compute rolling RPS, log loss and ECE on finished matches per league, compare them with the backtest confidence interval, and store them in a monitoring table; show them in the dashboard.
- Data-drift checks on key features (e.g. goals per match, Elo spread) per league, with thresholds in settings.yaml.
- Alerts (log + optional webhook URL from env) when rolling RPS leaves the backtest CI, drift thresholds are breached, or a scheduled job fails.
- Backups: nightly pg_dump to a mounted volume with retention; `make restore`; a documented restore drill.
- Hardening: resource limits in compose, restart policies, structured error handling in the scheduler, and a README with runbooks.
- Expansion proof: adding a third competition should only need a new settings.yaml entry, aliases and a tie-breaker rule. Prove it with a small fixture league used only in tests (do not add a real third league yet).

Acceptance tests (write first, show failing):

1. Monitoring: on a synthetic stream where predictions degrade, an alert fires; on a normal stream, none does.
2. Drift: a shifted feature distribution triggers the drift check.
3. Backup/restore: back up, wipe the database, restore, and row counts match.
4. Expansion: the fixture league runs through ingest -> predict with zero code changes outside config and aliases.
5. `make check` and `make e2e` still pass.

Finish with: `make check`, `make e2e` and `make monitor` outputs, the restore drill log, and a final review of the whole project: known limitations, leakage risks, and suggested next steps.

12.10 Working tips

- Stay in plan mode until you have read and approved the plan; ask for changes to the plan rather than to code.
- One phase per branch; merge only when make check and the phase's acceptance tests pass.
- After each phase, ask Claude Code for an independent review focused on data leakage and on tests that could pass trivially.
- Keep CLAUDE.md up to date: when you correct Claude twice about the same thing, add a rule.
- Review ingestion joins and date filters yourself; those bugs don't show up in metrics until it is too late.